CONSEQUA-X

Consequence-Grounded Causal Explainability for LLM Agents

Detailed Research and Engineering Brief

North-star question
Why did this agent produce this verified consequence?

Prepared for implementation with Codex
Research prototype specification
Version: October 2026

Executive Summary

Modern LLM agents can plan, use memory, retrieve information, call tools, delegate to sub-agents, and modify external systems. Existing observability systems increasingly answer what happened during a run, while recent research has begun addressing why an agent selected a particular action. Separately, work on reliable tool use shows that a tool response can diverge from the real external effect of that tool under timeouts, delayed visibility, partial commits, retries, or concurrent modification.

Consequa-X connects these layers. Its core thesis is that a faithful explanation of an autonomous agent action should distinguish (1) why the agent decided to act, (2) why it believed the action succeeded or failed, and (3) what consequence independently occurred in the external system.

Core research contribution

Connect causal trajectory attribution to independently verified external consequences. The explanation target is not merely a tool call or an agent-reported result; it is the verified real-world consequence and the chain that produced it.

Target causal structure

```
Trajectory -> Decision -> Action -> Tool Response -> Agent Belief
                    \-> External Effect -> Independent Verification
```

The project should remain an extension of Consequa rather than a separate generic observability product. Consequa verifies consequences. Consequa-X adds causal and counterfactual explanation over the trajectory that led to those consequences.

Document Map

1. Problem formulation and motivation
2. Research landscape and novelty boundary
3. Research gap, thesis, research questions, and hypotheses
4. System architecture and causal consequence graph
5. Data model, action identity, and provenance
6. Attribution, replay, and consequence-level causal effects
7. Belief divergence and failure taxonomy
8. ConseqBench benchmark design
9. Experimental plan, baselines, metrics, and ablations
10. Engineering architecture, APIs, repository layout, and build order
11. Testing, statistical requirements, validity threats, and publication criteria
12. Immediate Codex implementation specification
13. References

1. Problem Formulation

Traditional supervised explainability typically considers a model f(x) -> y and asks why y was produced. Autonomous agents operate over trajectories, interact with external systems, and form beliefs from tool responses. The explanatory object is therefore a process, not a single prediction.

```
H_t -> D_t -> A_t -> E_t -> R_t

H_t : historical trajectory/context
D_t : agent decision
A_t : intended action/tool call
E_t : execution attempt
R_t : response returned to the agent
```

This model is incomplete because the action also operates on external state:

```
S_t --E_t--> S_(t+1)

R_t may describe the effect, but R_t is not the effect itself.
```

Consequa introduces an independent verification channel. Let C be a developer-defined expected postcondition and V(S_(t+1), C) be a verifier over the external system of record. The resulting model has two partially independent channels:

```
Execution -> Tool Response -> Agent Belief
Execution -> External State -> Independent Verification
```

Key distinction
Never confuse what the agent said happened, what the tool reported happened, and what independently verifiably happened.

2. The Three Explanation Layers

2.1 Decision explanation

Question: Why did the agent choose this action? Candidate causes include the user request, system instruction, memory, retrieved documents, prior tool outputs, policy exceptions, or messages from other agents.

```
Why did RefundAgent call refund(order_42)?

Possible drivers:
- user request
- VIP memory
- retrieved exception policy
- order age
- sub-agent recommendation
- system instruction
```

2.2 Belief explanation

Question: Why did the agent believe the operation succeeded or failed? This is an epistemic explanation. The agent can reason consistently from misleading, missing, stale, or ambiguous evidence.

```
Agent belief: REFUND_FAILED
Observed evidence: HTTP request timed out
Interpretation: timeout treated as non-execution
Missing step: no independent state inspection
```

2.3 Consequence explanation

Question: What actually happened in the external system? Consequa supplies this layer through independent postcondition verification.

```
Tool response: TIMEOUT
Agent belief: REFUND_FAILED
Independent verification: refund rfnd_821 exists, amount=5000, status=processed
Actual consequence: REFUND_SUCCEEDED
```

3. Motivating Example

```
SYSTEM:
Refunds are normally allowed only within 30 days.

USER:
Please refund order #42.

MEMORY:
Customer is VIP.

RETRIEVAL:
VIP customers may receive discretionary exceptions.

TOOL:
get_order(42) -> age = 43 days

AGENT:
refund(order_42, INR 5,000)

REFUND API:
request times out

AGENT:
"Refund failed."

EXTERNAL SYSTEM:
refund actually committed successfully
```

A trace can show the sequence. Agentic attribution can identify that the VIP policy influenced the refund decision. A postcondition verifier can show that the refund actually occurred. Consequa-X must connect these into one explanation record and identify the divergence between response, belief, and reality.

```
VERIFIED CONSEQUENCE
INR 5,000 refund was processed.

DECISION CAUSE
Primary: VIP exception policy      0.71
Secondary: VIP customer memory     0.22
Conflicting evidence: order age=43 days

BELIEF CAUSE
Agent believed failure because refund API returned TIMEOUT.

REALITY
Provider state: refund exists, status=processed.

DIVERGENCE
Decision: refund intentionally selected
Execution: dispatched
Response: ambiguous
Belief: failed
Actual state: succeeded
Failure class: RESPONSE / BELIEF DIVERGENCE
```

4. Research Landscape and Novelty Boundary

The research must be positioned conservatively. Several components already exist independently, so the contribution is the connection between them rather than a claim to invent agent explainability or causal replay.

Area	State of the art / closest work	Novelty implication
Agent tracing	OpenAI Agents tracing; OpenTelemetry/OpenInference; Langfuse/LangSmith-style trajectory capture	Tracing is infrastructure, not the research contribution.
General agentic attribution	Qian et al. (2026) identify historical trajectory components and textual evidence influencing agent actions	Explaining why an agent acts is not by itself novel.
Failure attribution	Who&When, Who&When Pro, AgentRx	Failure localization alone is not novel.
Long-horizon attribution benchmarks	Long-Horizon Agent Trajectory Attribution (2026), >1,300 annotated trajectories	Use existing benchmarks where relevant.
Counterfactual causal replay	Causal Agent Replay (2026) performs interventions, reruns trajectories, reports confidence intervals, and uses Monte Carlo Shapley	Counterfactual replay itself is not novel.
Postcondition verification	Verified Tool Calls (2026); Callability Is Not Operability (2026)	Independent effect verification alone is not sufficient novelty.
Canonical action identity / attestation	CAVA (2026)	Action identity and provenance have related prior work; use them as design foundations.

Novelty boundary
Do not claim: first agent explainability framework; first counterfactual agent framework; first causal agent attribution system; first postcondition verification framework; first action-attestation system.

5. Proposed Research Gap

Existing work usually targets either the upstream question (why was an action selected?) or the downstream question (did an action produce its intended effect?). Consequa-X targets the connection between them.

```
UPSTREAM
Why did the agent decide?
        |
        v
DECISION -> ACTION -> EXECUTION / RESPONSE
        |
        v
DOWNSTREAM
What independently verified consequence occurred?
```

The explanatory target changes from H -> A to H -> A -> S' while explicitly modeling A -> R separately from A -> S'. This yields an Action-Response-Effect causal model.

Primary research question
Can agent explanations become more faithful and diagnostically useful when causal trajectory attribution is grounded in independently verified external consequences rather than stopping at the selected tool call or reported result?

6. Central Thesis, Research Questions, and Hypotheses

Central thesis: agent explanations that concern real-world actions should be grounded in both upstream causal trajectory evidence and independently verified downstream external state.

ID	Research question
RQ1	Can independently verified world state improve explanation faithfulness when tool responses and actual effects disagree?

RQ2	Can the framework correctly distinguish whether a problematic outcome originated from decision, execution, response/observation, belief, verification, policy, or an external actor?
RQ3	Can trajectory interventions identify historical components that causally influenced the decision leading to a verified consequence?
RQ4	Can the framework explain why an agent formed an incorrect belief about whether its action succeeded?
RQ5	How does consequence-grounded attribution perform when causal evidence appears many steps before the final action?
RQ6	When multiple agents contribute to a consequence, can the system recover responsibility across agents? (secondary/stretch)

ID	Hypothesis
H1	Under atomic and perfectly reliable tools, consequence-grounded attribution and ordinary trajectory attribution should perform similarly.
H2	Under response/effect divergence, trajectory-only methods will produce more incorrect explanations than consequence-grounded methods.
H3	Explicit decision, response, belief, and effect channels will improve failure-layer localization.
H4	Intervention-based attribution will outperform pure LLM trace summarization for recovering planted causal factors.
H5	Ground-truth postcondition verification will improve explanation calibration because the explanation no longer depends on the agent's self-reported interpretation of tool success.

7. Target Contributions

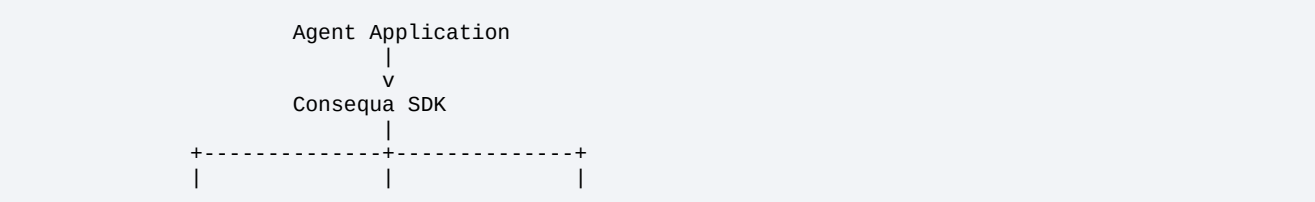
- Action-Response-Effect causal model: a formal representation separating decision, execution, response, belief, external effect, and verification.
- Causal Consequence Graph (CCG): a framework-independent graph linking trajectory events to independently observed environmental effects.
- Consequence-grounded attribution: intervention-based attribution whose target can be a verified external consequence rather than only action selection.
- ConseqBench: a benchmark with known upstream decision causes, non-atomic execution failures, agent beliefs, external effects, and ground-truth causal chains.

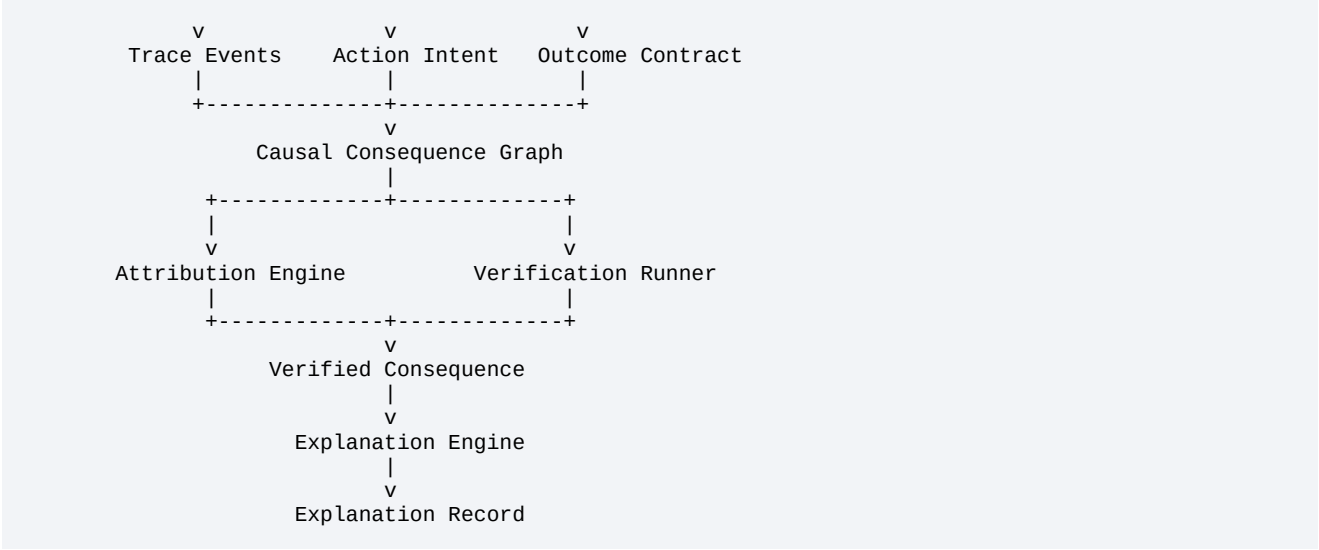
8. Non-Goals

- Do not build a generic observability platform.
- Do not replace LangSmith, Langfuse, OpenTelemetry, or OpenInference.
- Do not treat an LLM-generated self-explanation as ground truth.
- Do not require hidden chain-of-thought; operate on externally available trajectory components.
- Do not make causal claims from a single nondeterministic rerun.
- Do not repeatedly execute consequential production writes during counterfactual analysis.

9. System Architecture

Consequa remains the runtime foundation. Consequa-X adds an explanation and research layer around its action registration, outcome contracts, trace events, and verification results.





10. Implementation Language Strategy

Preserve existing Consequa runtime components in TypeScript. Add a Python research package for benchmark generation, statistical analysis, replay experiments, causal estimators, and plotting. Initially exchange trace and experiment data through JSON/JSONL rather than introducing RPC complexity.

Layer	Recommended language	Responsibilities
Runtime / SDK	TypeScript	Consequa SDK, tracing, action contracts, verifiers, dashboard integration
Research package	Python	counterfactual experiments, benchmark generation, metrics, statistical analysis
Boundary	JSON / JSONL	portable exchange between runtime traces and research tooling

11. Causal Consequence Graph (CCG)

The CCG is the central intermediate representation. It should import standard OpenTelemetry/OpenInference-style records where possible and enrich them rather than inventing an entirely incompatible tracing schema.

Node types:

```
USER_INPUT
SYSTEM_INSTRUCTION
AGENT_MESSAGE
AGENT_DECISION
MEMORY_READ
MEMORY_WRITE
RETRIEVAL
TOOL_CALL
TOOL_RESPONSE
HANDOFF
GUARDRAIL
ACTION_INTENT
ACTION_DISPATCH
EXTERNAL_OBSERVATION
POSTCONDITION
VERIFICATION
EXTERNAL_EFFECT
POLICY
FINAL_OUTCOME
```

Edge types:

```
TEMPORAL
DATA_DEPENDENCY
CONTROL_DEPENDENCY
RETRIEVAL_DEPENDENCY
MEMORY_DEPENDENCY
DELEGATION
EXECUTION
RESPONSE
EFFECT
VERIFICATION
CAUSAL_ATTRIBUTION
CONFLICT
```

12. Core Data Model

```
type EventType =
  | "user_input"
  | "system_instruction"
  | "agent_decision"
  | "agent_message"
  | "memory_read"
  | "memory_write"
  | "retrieval"
  | "tool_call"
  | "tool_response"
  | "handoff"
  | "action_intent"
  | "action_dispatch"
  | "external_observation"
  | "postcondition"
  | "verification"
  | "external_effect";

interface AgentEvent {
  id: string;
  traceId: string;
  parentIds: string[];
  type: EventType;
  timestamp: string;
  agentId?: string;
  input?: unknown;
  output?: unknown;
  metadata?: Record<string, unknown>;
}
```

```
interface ConsequaAction {
  actionId: string;
  traceId: string;
  actor: string;
  name: string;
  arguments: Record<string, unknown>;
  intentRegisteredAt: string;
  dispatchedAt?: string;
  idempotencyKey?: string;
  expectedOutcome: OutcomeContract;
}

interface OutcomeContract {
  contractId: string;
  actionName: string;
  predicate: string;
  verifier: VerifierDefinition;
  timeoutMs: number;
  pollingIntervalMs?: number;
}
```

```
type VerificationStatus =
  | "satisfied"
  | "violated"
  | "unknown"
  | "timeout"
  | "reversed";

interface VerificationResult {
  status: VerificationStatus;
  observedAt: string;
  evidence: Evidence[];
  confidence?: number;
}
```

```
interface ConsequenceExplanation {
  actionId: string;
  verifiedConsequence: VerificationResult;
  decisionAttribution: AttributionResult[];
  beliefAttribution: AttributionResult[];
  divergence:
    | "none" | "decision" | "execution" | "response"
    | "belief" | "verification" | "external_interference";
  counterfactuals: CounterfactualResult[];
  causalChain: string[];
  confidence: number;
}
```

13. Action Identity and Provenance

A postcondition holding does not prove that the agent caused it. Another actor may have produced the same state transition. Every action should therefore preserve causal identity and provenance as strongly as the provider permits.

```
action_id
idempotency_key
provider request ID
transaction ID
entity ID
timestamp
expected parameters
trace ID
```

Where external providers support idempotency keys or metadata tags, propagate the Consequa action ID. This creates a stronger evidentiary chain from intent to dispatch to provider operation to externally observed effect.

```
cq_action_8192
    -> provider idempotency / metadata key
    -> provider operation
    -> provider transaction ID
    -> independent verifier
```

14. Attribution Engine

14.1 Baseline 0: LLM explanation

Give the trajectory to an LLM and ask which event most influenced the target action. This is intentionally weak and should be used as a baseline rather than treated as causal evidence.

14.2 Baseline 1: Leave-one-out trajectory attribution

For each candidate historical component h_i, estimate the shift in action probability after removing or neutralizing the component:

```
Delta_i = P(A | H) - P(A | H without h_i)
```

```
Original: refund selected 18/20
Remove VIP memory: refund selected 13/20
Remove VIP policy: refund selected 2/20

Interpretation: VIP policy has the strongest observed influence.
```

14.3 Intervention types

```
remove(event)
replace(event, neutral_value)
mask(event)
modify(event, altered_value)
remove_agent(agent_id)
replace_tool_output(...)
remove_memory(...)
replace_retrieval(...)
```

Interventions must preserve a valid agent state. Arbitrary deletion is not acceptable when it creates an impossible or malformed conversation history.

15. Replay Modes

15.1 Production shadow replay

Never repeatedly execute consequential external writes. Fork before the decision, replay or mock recorded tool reads, and rerun the decision. Use this to estimate what caused the original decision. Consequa independently verifies what occurred in the real world.

```
historical trace
    -> fork before decision
    -> mock/replay recorded tools
    -> rerun decision
    -> estimate decision attribution
```

15.2 Research benchmark replay

In isolated simulated environments, execute complete counterfactual trajectories. The verified final postcondition can then be used directly as the causal target variable.

```
intervention
    -> agent rerun
    -> simulated tool execution
    -> simulated external state
    -> postcondition verification
```

16. Consequence-Level Causal Effect

For event h_i, define a consequence-level effect in benchmark mode as:

```
CE(h_i) = P(V=1 | H) - P(V=1 | do(H without h_i))

V = verified target consequence
```

This differs from measuring only whether the agent selected an action. For harmful consequences, the target can be a verified harm indicator. Estimate effects through repeated replay and report uncertainty.

```
VIP_POLICY
Original verified refund rate:        0.90
Counterfactual verified refund rate:  0.08
Estimated consequence effect:        +0.82
95% CI:                              [0.67, 0.92]
```

17. Belief Divergence

Represent the agent belief B_t about action outcome separately from independently verified outcome V_t. Define a divergence indicator D_t = 1[B_t != V_t]. This makes epistemic failure a first-class research object.

```
Agent says SUCCESS  | Reality says FAILURE
Agent says FAILURE  | Reality says SUCCESS
Agent says SUCCESS  | Reality says PARTIAL
Agent says FAILURE  | Reality says UNKNOWN
```

18. Failure Taxonomy

Failure class	Description
DECISION	Agent chose the wrong action.
PARAMETER	Correct action type, wrong arguments.
EXECUTION	Correct decision but action failed to execute.
PARTIAL_EFFECT	Only some expected effects occurred.
RESPONSE	Tool response did not faithfully represent the effect.
BELIEF	Agent misinterpreted available evidence.
VISIBILITY	Correct effect was not yet observable.
DUPLICATION	Retry created repeated side effects.
REVERSAL	Initially valid consequence later reversed.
EXTERNAL_INTERFERENCE	State changed because of another actor.
POLICY	Action violated an independent policy.
VERIFIER	Verification mechanism was incomplete or incorrect.

The system should distinguish the causal driver from the failure layer. The event that caused a decision can be different from the layer at which the system ultimately failed.

19. ConseqBench

ConseqBench is a controlled benchmark for attribution from trajectory components to independently verified external consequences under realistic execution uncertainty. Avoid live payment infrastructure; use deterministic local simulators with explicit ground truth.

19.1 Initial domains

Domain	Representative tools	Representative postconditions
Payments / refunds	create_payment, refund_payment, get_payment, get_refund	refund exists; correct amount; correct payment; processed status
CRM / customer operations	create_customer, update_customer, send_message, get_customer, get_messages	customer created once; status correct; exactly one message delivered
Inventory / purchase orders	get_inventory, create_purchase_order, approve_purchase_order, cancel_purchase_order, get_purchase_order	correct quantity; supplier; approval status; no duplicate PO

19.2 Fault injection

```
F0  clean success
F1  fail-before-dispatch
F2  timeout-before-commit
F3  timeout-after-commit
F4  response success but no mutation
F5  delayed visibility
F6  partial mutation
F7  duplicate effect
F8  stale read
F9  concurrent external modification
F10 reversal after apparent success
F11 wrong-target mutation
F12 wrong-parameter mutation
```

19.3 Upstream decision perturbations

```
D0 clean instruction
D1 misleading memory
D2 poisoned retrieval
D3 conflicting system/user instruction
D4 misleading sub-agent message
D5 incorrect tool observation
D6 stale memory
D7 irrelevant distractor context
D8 policy exception
```

Together these produce controlled chains from an upstream causal factor to a decision, action, execution fault, belief, and verified consequence.

19.4 Ground-truth labels

```
gold_decision_driver
gold_decision_step
gold_action
gold_parameters
gold_execution_fault
gold_actual_mutation
gold_effect_source
gold_agent_belief
gold_postcondition
gold_failure_layer
gold_causal_chain
```

19.5 Dataset size

Start with approximately 500-1,000 development runs. A first full benchmark can target roughly 3 domains x 5 task templates per domain x 8 important execution conditions x 5 upstream conditions x 10 seeds, or approximately 6,000 runs. Scale only after the pipeline is stable.

20. Evaluation Baselines

Baseline	Description
B0 Trace-only LLM judge	Full trace -> LLM -> explanation.
B1 Recency heuristic	Attribute to the most recent relevant step.
B2 Leave-one-out agentic attribution	Measure changes in action selection under component interventions.
B3 Causal Agent Replay	Use/reproduce CAR where applicable rather than re-claiming its method.
B4 Consequa verification only	Determine what happened externally without upstream explanation.

B5 Consequa-X	Joint decision attribution + belief divergence + verified effect + causal consequence graph.

21. Metrics

Category	Metrics
Decision attribution	Top-1 attribution accuracy; Top-3 attribution recall; MRR
Attribution chain	Precision; recall; F1; edge F1
Failure localization	Failure layer accuracy; failure step accuracy; agent attribution accuracy
Consequence verification	Outcome accuracy; false verification rate; unknown rate; reversal detection rate
Consequence-grounded explanation	Verified Consequence Attribution Accuracy; Divergence Localization Accuracy; False Explanation Rate
Counterfactual quality	Counterfactual Fidelity
Calibration	Brier score; expected calibration error where appropriate

Important metric
False Explanation Rate: among runs where tool response and external state disagree, how often does a method explain the incorrect response instead of the actual verified state?

22. Experimental Plan

22.1 Primary experiment: clean vs non-atomic tools

Hold tasks and upstream trajectories constant. Compare trajectory-only attribution with consequence-grounded attribution under atomic tools and non-atomic tools. Expected result: similar performance under clean tools, with Consequa-X improving substantially when response/effect divergence is introduced.

22.2 Timeout after commit

```
agent decides refund
    -> refund commits
    -> response lost
    -> agent sees timeout

Ground truth: refund succeeded
```

Measure whether each explanation method incorrectly reports failure or correctly grounds itself in verified state.

22.3 False success

```
tool returns SUCCESS
    -> external mutation absent
```

Measure whether trajectory-only methods implicitly treat the response as reality.

22.4 Partial effects

```
Create supplier
Update inventory
Create PO

Only supplier + inventory succeed.
Expected diagnosis: PARTIAL_EFFECT
```

22.5 Long-horizon causal driver

Plant the true causal driver 15-30 steps before the action and compare recency, LLM attribution, leave-one-out, and causal replay methods.

22.6 External actor confounding

```
agent attempts to mark invoice PAID
    -> agent operation fails
    -> separate process marks invoice PAID
    -> verifier observes PAID
```

A naive postcondition check says success, but the agent did not cause the state transition. Use action identity, timestamps, transaction metadata, and provenance to separate postcondition satisfaction from causal responsibility.

22.7 Multi-agent responsibility (later phase)

```
ResearchAgent -> Planner -> PolicyAgent -> Executor -> verified consequence
```

Inject an error upstream but allow the consequence to appear only at the Executor. Evaluate whether the system recovers the cross-agent causal chain.

23. Ablations

```
Full system
- no external verifier
- no counterfactual attribution
- no response/effect separation
- no action identity
- no uncertainty estimation
- no belief modelling
```

24. Explanation Output Standard

Expose a standardized Agent Consequence Explanation Card. It should be both machine-readable and human-readable.

```
ACTION
refund(order_42, INR 5,000)

ACTOR
RefundAgent

DECISION DRIVERS
VIP exception policy       0.78
VIP memory                 0.31
customer request           0.24

CONFLICTING EVIDENCE
Order age: 43 days
Normal refund limit: 30 days

COUNTERFACTUAL
Without VIP exception: refund selection 91% -> 8%

TOOL RESPONSE
TIMEOUT

AGENT BELIEF
REFUND_FAILED

VERIFIED CONSEQUENCE
refund_id: rfnd_8821
amount: INR 5,000
status: processed

DIVERGENCE
```

```
Response channel: ambiguous
Effect channel: success
Agent belief: incorrect

FAILURE LAYER
BELIEF / RESPONSE

CAUSAL CHAIN
VIP exception retrieval -> refund decision -> dispatch -> provider commit
-> lost response -> incorrect agent belief -> independent verification
```

25. Proposed SDK Experience

```
const refund = await consequa.action({
  name: "refund",
  input: { paymentId, amount: 5000 },
  outcome: {
    verifier: "razorpay.refund",
    expect: { paymentId, amount: 5000, status: "processed" },
  },
});

await refund.execute(() =>
  razorpay.refunds.create({
    payment_id: paymentId,
    amount: 5000,
  })
);

const explanation = await consequa.explain(refund.id);
```

```
{
  consequence: { status: "satisfied", evidence: [...] },
  decision: { primaryDriver: "retrieval_7", attribution: 0.78 },
  belief: { status: "incorrect", driver: "timeout_response_9" },
  divergence: "belief",
  counterfactuals: [...]
}
```

26. Research API

```
trace = ConsequaTrace.load("trace_001.json")

target = VerifiedConsequence(
    action_id="refund_1",
    predicate="refund.status == 'processed'"
)

result = explain(
    trace=trace,
    target=target,
    method="leave_one_out",
    samples=20,
)
```

```
cf = trace.fork(before="decision_12")
cf.remove("memory_4")
result = runner.run(cf, samples=20)
```

27. Repository Structure

```
consequa/
|
```

```
+-- apps/
|   +-- dashboard/
|   +-- docs/
|   +-- ingest-api/
|
+-- packages/
|   +-- sdk/
|   +-- contracts/
|   +-- verifiers/
|   +-- tracing/
|   +-- consequence-graph/
|   +-- explanation-schema/
|
+-- runner/
|
+-- research/
|   +-- pyproject.toml
|   +-- consequa_x/
|   |   +-- schema/
|   |   +-- graph/
|   |   +-- attribution/
|   |   +-- replay/
|   |   +-- verification/
|   |   +-- metrics/
|   |   +-- experiments/
|   |
|   +-- conseqbench/
|   |   +-- environments/
|   |   |   +-- payments/
|   |   |   +-- crm/
|   |   |   +-- inventory/
|   |   +-- faults/
|   |   +-- tasks/
|   |   +-- labels/
|   |
|   +-- baselines/
|   |   +-- llm_judge/
|   |   +-- recency/
|   |   +-- loo/
|   |   +-- car/
|   |
|   +-- scripts/
|       +-- generate_dataset.py
|       +-- run_experiment.py
|       +-- evaluate.py
|       +-- make_figures.py
|
+-- examples/
    +-- refund-agent/
    +-- crm-agent/
    +-- inventory-agent/
```

28. Build Order

Phase	Deliverable
0 - Research skeleton	research/ directory, README, RQs, schemas, experiment config. No UI.
1 - Deterministic world	One payments/refunds simulator; clean, timeout-after-commit, false-success, delayed-visibility, partial-effect faults; ground-truth log.
2 - Consequa verifier	OutcomeContract, Verifier, VerificationResult; unit tests for response != world state.
3 - Trace representation	Canonical AgentEvent and Causal Consequence Graph import/build.
4 - Baselines	Recency, LLM judge, leave-one-out.
5 - Replay	Snapshot, fork, remove, replace, rerun using simulator snapshots.
6 - Consequence attribution	Target verified consequences rather than only action selection.
7 - Epistemic layer	Represent agent belief, tool response, verified reality; detect mismatches.

8 - ConseqBench	Add CRM and inventory; generate labeled runs.
9 - Dashboard	Visualize trajectory, graph, verified outcome, attribution, belief divergence, counterfactuals.

29. Tests Required

```
test_clean_success
test_timeout_after_commit
test_timeout_before_commit
test_false_success
test_partial_effect
test_delayed_visibility
test_duplicate_action
test_external_actor_confounding
test_reversal
test_wrong_parameter

test_remove_true_driver_changes_decision
test_remove_irrelevant_event_does_not_change_decision
test_long_range_driver
test_multiple_interacting_causes

test_action_to_response_edge
test_action_to_effect_edge
test_effect_to_verification_edge
test_response_and_effect_can_disagree
```

30. Statistical Requirements

Agent behavior is probabilistic. Never infer causality from one rerun. Begin with at least 20 repeated samples for each stochastic intervention, then increase sample counts when variance is high. Report point estimates, confidence intervals, sample counts, and seeds/configuration needed for reproducibility.

Avoid categorical language such as "caused action" when evidence is weak. Prefer numerical estimates or calibrated labels such as strong attributable influence, moderate attributable influence, and insufficient evidence.

31. Threats to Validity

Threat	Mitigation
Simulator realism	Add a realistic provider-style emulator and optionally one real sandbox integration after controlled experiments.
Intervention validity	Use typed interventions and neutral replacements rather than malformed history deletion.
Model nondeterminism	Repeated sampling and confidence intervals.
Verifier correctness	Use deterministic, hand-written benchmark verifiers.
External causality	Action IDs, correlation metadata, provider transaction IDs, explicit confound experiments.
Generalization	State that payment/CRM/inventory results may not generalize to browser, robotics, or physical agents.

32. Kill Criteria and Minimum Publishable Result

Be willing to substantially modify or abandon the central claim if trajectory-only attribution works equally well under high response/effect divergence, if independent verification adds no measurable explanatory value, or if external action identity cannot be established reliably enough for causal claims.

A credible first paper does not need dozens of frameworks or integrations. It needs a clear problem definition, formal model, controlled benchmark, strong ground truth, multiple baselines, statistically defensible experiments, ablations, and one convincing end-to-end demo.

```
Minimum target:
- 3 environments
- approximately 3,000-6,000 trajectories
- 8-12 execution conditions
- 4-6 upstream perturbation types
- at least 4 attribution baselines
- full ablation study
- one reproducible demo
```

33. Proposed Paper Structure

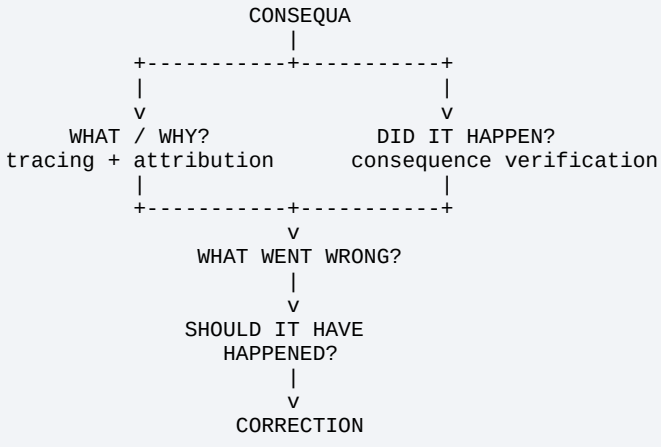
1. Introduction
2. Background: LLM agents, trajectory attribution, causal replay, tool reliability, postcondition verification
3. Motivation: response/effect divergence and why tracing is insufficient
4. Problem formulation: decision channel, response channel, effect channel, verified consequence
5. Consequa-X: CCG, verifier, attribution, belief divergence
6. ConseqBench: environments, fault injection, ground truth
7. Experimental setup
8. Results
9. Ablations
10. Limitations and threats to validity
11. Related work
12. Conclusion

One-sentence paper pitch

LLM-agent explanations typically stop at what action was selected, while real-world tools can report outcomes that differ from the effects they actually produce; Consequa-X connects causal trajectory attribution with independently verified external state to explain not only why an agent acted, but what consequence its action verifiably produced and where decision, belief, execution, and reality diverged.

34. Relationship to Consequa

Consequa-X should be developed as the explainability layer of Consequa, not as an unrelated repository unless experimentation later shows that separation is necessary.



Long-term product implication: an evidence and debugging layer for consequential autonomous-agent actions. A user should be able to inspect intent, causal drivers, tool response, agent belief, actual state change, verification evidence, provenance, policy compliance, and meaningful counterfactuals in one record.

35. Immediate Codex Implementation Specification

Do this first
Do not build the dashboard first. Inspect the existing Consequa repository, preserve its architecture, and implement the smallest end-to-end research slice that proves response/effect/belief divergence can be represented and verified.

Create a new research/ package containing:

1. Canonical research schemas.
2. A deterministic simulated payment/refund environment.
3. Fault injection for clean success, timeout-after-commit, false success, delayed visibility, and partial effect.
4. Outcome contract abstraction.
5. Read-only postcondition verifier.
6. Ground-truth execution log.
7. Causal Consequence Graph builder.
8. Recency attribution baseline.
9. Leave-one-out attribution interface.
10. Pytest coverage for all core divergence cases.

The first end-to-end test must demonstrate the following exact pattern:

```
1. Agent chooses refund.
2. Refund API times out.
3. Simulator commits refund.
4. Agent records failure belief.
5. Consequa verifier independently discovers refund.
6. Graph contains:
   trajectory -> refund decision -> dispatch -> timeout response -> failure belief
   and separately:
   dispatch -> actual refund effect -> verification satisfied
7. Final diagnosis: BELIEF / RESPONSE DIVERGENCE
```

Only after this test passes should additional domains, richer causal replay, LLM-driven agents, multi-agent responsibility, or UI work begin.

36. Codex Guardrails

- Do not rewrite the existing Consequa runtime unless required by a demonstrated incompatibility.
- Prefer adapters around existing tracing and verification abstractions.
- Use deterministic local simulation before any real provider integration.
- Every injected fault must be observable in a ground-truth execution log.
- Every verification result must contain evidence, not only a boolean.
- Keep tool response, agent belief, and external effect as separate fields.
- Do not label observational correlation as causal attribution.
- Keep experiments reproducible with explicit seeds and configuration files.
- No production side-effect should be re-executed during counterfactual analysis.
- Add tests before dashboard work.

37. Research Principle

Principle 1
Never confuse what the agent said happened, what a tool reported happened, and what independently verifiably happened.

Principle 2
Never confuse a plausible explanation with a causally supported explanation.

Consequa addresses the first principle. Agent attribution research addresses the second. Consequa-X connects them.

North-star question: Why did this agent produce this verified consequence?

References and Closest Prior Work

[R1] OpenAI Agents SDK - Tracing. https://openai.github.io/openai-agents-python/tracing/

[R2] OpenInference Specification - semantic conventions for AI/LLM tracing on OpenTelemetry. https://arize-ai.github.io/openinference/spec/

[R3] Qian, C. et al. (2026). The Why Behind the Action: Unveiling Internal Drivers via Agentic Attribution. arXiv:2601.15075. https://arxiv.org/abs/2601.15075

[R4] Shah, J. (2026). Causal Agent Replay: Counterfactual Attribution for LLM-Agent Failures. arXiv:2606.08275. https://arxiv.org/abs/2606.08275

[R5] Chen, J. et al. (2026). Long-Horizon Agent Trajectory Attribution: A Unified Benchmark and Fine-Grained Annotation Framework. arXiv:2608.06909. https://arxiv.org/abs/2608.06909

[R6] Zhang, S. et al. (2025). Which Agent Causes Task Failures and When? On Automated Failure Attribution of LLM Multi-Agent Systems. ICML 2025 / PMLR 267. https://proceedings.mlr.press/v267/zhang25cq.html

[R7] Liu, J. et al. (2026). Who&When Pro: Can LLMs Really Attribute Failures in AI Agents? arXiv:2607.09996. https://arxiv.org/abs/2607.09996

[R8] Barke, S. et al. (2026). AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475. https://arxiv.org/abs/2602.02475

[R9] Mansoor, I. K., Phadke, A., Rana, P. (2026). Verified Tool Calls Improve LLM Agent Reliability Under Non-Atomic Failures. arXiv:2608.02645. https://arxiv.org/abs/2608.02645

[R10] Wang, Z. (2026). Callability Is Not Operability: Controlled Interface Interventions for LLM Agents. arXiv:2608.23628. https://arxiv.org/abs/2608.23628

[R11] Wang, Z. (2026). CAVA: Canonical Action Verification and Attestation for Runtime Governance of Agentic AI Systems. arXiv:2607.13716. https://arxiv.org/abs/2607.13716

Note: several 2026 works above are preprints. Before submission, re-check publication status, versions, authorship, and related work for any newer papers that materially overlap with the final method.

Appendix A - Short Handoff Prompt for Codex

```
You are working in the existing Consequa repository. Read this research brief as the project
specification.

Your first goal is NOT to build a UI. Build the minimum end-to-end research slice that proves the
Action-Response-Effect model.

Required first milestone:
- create research/ package
- deterministic refund simulator
- action intent + outcome contract
- read-only postcondition verifier
- fault: timeout-after-commit
- explicit agent belief field
- Causal Consequence Graph
- ground-truth log
- pytest coverage

The demo must show:
agent chooses refund -> provider commits -> response times out -> agent believes failure ->
independent verifier finds success -> diagnosis = BELIEF / RESPONSE DIVERGENCE.

Preserve the current Consequa architecture. Do not add unnecessary frameworks. Do not execute
real external writes during replay. Keep response, belief, and external effect as separate
objects. Once the milestone passes, implement baselines and ConseqBench incrementally.
```